

How AgroScan works — from a leaf photo to farmer advice

This booklet is for anyone. You do not need to be a programmer. Hard words are explained the first time they appear. The numbers and file names are the real ones from the AgroScan code.

The 5-year-old version: AgroScan is a plant doctor that lives in a phone. You show it a picture of a leaf. First it checks “is this even a leaf?” Then “which plant?” Then “what is wrong?” After that it opens a real Bangladesh farming book — not a guessing robot — and tells you what to do. If you type a question, it tries that book first. Only leftover hard questions go to a talking robot named Qwen, and Qwen is only allowed to use pages we fetched from the book.

What you will learn

1. The big picture: phone, website, and the computer that does the thinking.
2. The three picture helpers (leaf gate, crop, disease).
3. Where the training photos came from and how they are folded into folders.
4. How we “teach” the helpers (training) and what files get saved.
5. How the winner disease is picked (no averaging).
6. The advice pack: doses, sprays, 16123.
7. Chat, RAG, and Qwen — who talks, who is not allowed to invent medicine.
8. Where everything lives on disk, and how to run it.

One sentence for grown-ups

AgroScan is a three-level vision pipeline plus a pack-first chat API, served by FastAPI, for Bangladeshi farmers (Bangla by default).

Written from the running code in `I:\Agroscan\Agroscan` · August 2026 · Python package name: `agroscan`

The big picture: a shop window and a kitchen

Simple: The website is a shop window. You look at it and press buttons. The real cooking happens in the kitchen — a Python program on a computer (often a PC with an RTX 3060 graphics card). The window never “thinks.” It only shows pictures and sends the photo to the kitchen.

The kitchen is a **FastAPI** server. That is a small web program that waits for requests. The same kitchen feeds:

- the website in a browser
- the PWA (install-to-home-screen app)
- the Android app (Capacitor wrapper around the same website)

Two ways a farmer talks to AgroScan

1. A photo
Send a leaf picture to `POST /api/predict` or `POST /api/chat/vision`. The three picture helpers run. Then AgroScan writes advice from the Bangladesh pack. Qwen may add a short wrap, but only if it does not start repeating itself.

2. A typed question
Send text to `POST /api/chat`. If you named a disease, you get the pack — no robot. If the name is fuzzy, AgroScan asks which crop. Hellos and 16123 stay on a simple knowledge base. Only leftover open questions go through RAG + Qwen.

What “the kitchen” is made of

Piece	Folder	Job in kid words
Python core	<code>agroscan/</code>	Eyes, book, shop, chat
API	<code>backend/main.py</code>	The waiter who takes orders
Website	<code>web/</code>	The shop window
Android	<code>mobile/</code>	Same window, inside a phone app
Picture brains	<code>models/*.pt</code>	What we taught the eyes to remember
Training photos	<code>Datasets/</code>	The flashcards we used in school
Advice book	<code>data/agroscan/</code>	Verified doses, not guesses
Talking robot files	<code>llm_data/</code>	Optional extra training for Qwen

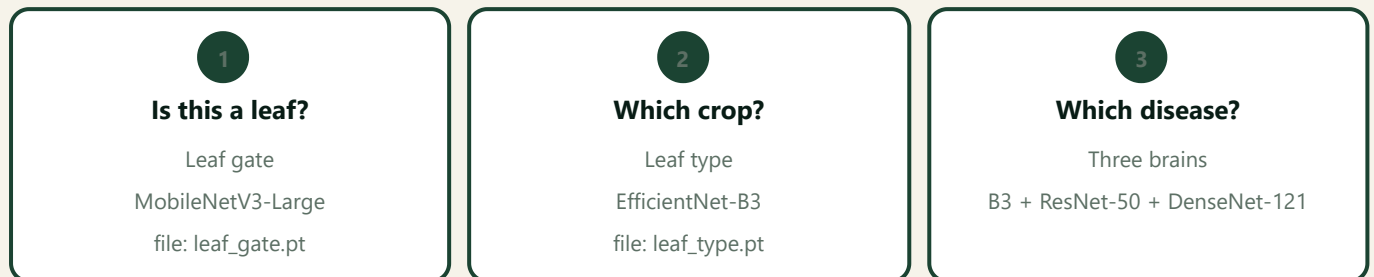
Start the kitchen with `start_server.bat` or:

```
python -m uvicorn backend.main:app --host 0.0.0.0 --port 8000
```

Then open **http://localhost:8000**.

Three questions, in order

We do not jump straight to “what disease is this?” That would be like a doctor shouting a medicine name before checking you are even a person. AgroScan asks three questions.



Simple: First we make sure you did not photograph a shoe. Then we find out if the plant is rice or tomato. Only then do we name the spots. If we named the disease first, a tomato photo could be called “rice blast” because the spots look a bit alike.

Level 1 — leaf gate

A small, fast network called **MobileNetV3-Large** (a compact neural net — a pile of number-crunching layers that learned patterns from pictures). It answers leaf vs not-leaf. If the “this is a leaf” score is below **0.50**, the app stops. Status:

`not_leaf`.

Why a gate at all? Disease models will always pick *some* class, even for a selfie. The gate is the “are we in the right universe?” check.

Level 2 — which crop

EfficientNet-B3 names the plant (rice, tomato, mango, ...). If it is less than **90% sure**, the farmer must pick from the top 3. Status: `need_crop`. That is kinder than a wrong crop, because the next step *locks* diseases to that crop.

Level 3 — which disease

Three different networks look at the same photo. Each outputs a list of scores, one per disease class (labels look like `Rice__Blast`). Scores are **masked** to the Level-2 crop: tomato classes only if the crop is tomato.

Important: AgroScan does *not* average the three brains. It picks the **single model with the highest confidence**. Think: three doctors write an answer; we take the one who is most sure — we do not blend their handwriting.

If confidence is under **0.45** the result is flagged uncertain. Under **0.80** the UI may ask for a clearer photo. Code:

`agroscan/infer.py`.

The flashcards: how the picture data is organised

Simple: A neural net learns like a child with a huge stack of labelled photos. “This is a leaf.” “This is rice.” “This is rice blast.” Folders *are* the labels. If a photo sits in `Datasets/train/Rice__Blast/`, the teacher tells the net “the answer is rice blast.”

We split photos into three piles so we do not cheat on the test:

- **train** — the homework (the net looks at these while learning)
- **valid** — the practice quiz after each round (we pick the best quiz score to save)
- **test** — the final exam (held back so we do not tune until it only memorizes homework)

Three photo schools (matching the three questions)

School	Folder	What is in it
Level 1	<code>Datasets/leaf_gate/</code>	Two classes: <i>leaf</i> and <i>non_leaf</i> . Non-leaf photos are capped at 1.5× the leaf count so fashion pictures do not drown the leaves.
Level 2	<code>Datasets/leaf_type/</code>	About 27 crop folders (Apple, Rice, Tomato, Mango, ...). Mango variety photos live under Mango here. They are <i>not</i> copied into Level 3 disease folders.
Level 3	<code>Datasets/train valid test/</code>	About 123 disease classes (PlantVillage-style names plus Bangladesh crops). Roughly 241k / 30k / 30k images in train / valid / test.

Where those photos originally came from

Training default is **combined** (`DEFAULT_DISEASE_SOURCE`). Sources that were merged into the unified splits include:

- **PlantVillage** — classic 38 leaf-disease classes (apple scab, tomato late blight, ...), raw + augmented
- **Bangladesh crop disease** — extra local classes
- **PlantDoc** — messier field photos (harder, more like a real phone picture)
- **Archive crop disease + Mangifera 2012** mango varieties (varieties stay in leaf_type, not disease)
- **Bangladeshi Leaf Disease Detection** zip (extracted on first train if needed)

Scripts under `scripts/` used Windows hard links so the same photo file can appear in train/valid/test trees without copying gigabytes twice.

Other data that is *not* pictures: crop CSVs in `Datasets/tabular/crops/`, Q&A text in `Datasets/11m/`, maps in `Datasets/geospatial/`.

How we teach the picture helpers

Simple: Show a photo. The net guesses. If it is wrong, we nudge its millions of tiny numbers (weights) a little so next time it is less wrong. Do this thousands of times. One full walk through the homework pile is an **epoch**. We use a graphics card (GPU) because it is like 3,000 calculators at once.

We do not train from a blank brain. The networks start from **ImageNet** — they already know generic shapes (edges, textures) from 1.2 million everyday photos. We then fine-tune them on leaves. That is why training can finish in hours, not months.

One command trains all three levels

```
python -m agrosan.train_all
```

Or one class at a time: `train_leaf`, `train_crop`, `train_disease --arch efficientnet_b3`.

Level	Network	Photo size	Epochs	Batch	Learning rate
1 Leaf	MobileNetV3-Large	224×224	4	64	0.001
2 Crop	EfficientNet-B3	300×300	6	32	0.001
3 Disease ×3	B3, ResNet-50, DenseNet-121	300 / 224 / 224	6 each	32	0.001

Batch = how many photos in one nudge. **Learning rate 0.001** = how big each nudge is. Too big = chaotic; too small = forever.

Tricks so rare diseases are not ignored

- **Focal loss** (gamma = 2) — if the net is already sure and right, we barely poke it. If it is confidently wrong on a rare class, we poke harder. Kid version: the teacher spends more time on the questions you keep missing.
- **Balanced sampler** — rare folders get shown more often, so “rice blast” does not get shouted down by a huge “healthy tomato” pile.
- **Cosine learning-rate schedule + mixed precision (AMP)** — faster, kinder to a 12 GB RTX 3060.
- **Seed 42** — so shuffles are repeatable.

What gets saved in `models/`

File	What it is
<code>leaf_gate.pt</code> <code>leaf_type.pt</code> <code>disease_*.pt</code>	Best quiz (validation) weights — this is what the app loads
<code>*.train.pt</code>	Full snapshot: model + optimizer + scheduler + AMP scaler + epoch, for <code>--resume</code> after Ctrl+C

On this Windows PC, `NUM_WORKERS = 0` so photo loading stays in the main process. Extra worker processes import CUDA and can crash with “paging file too small.”

After school: using the brains, then opening the book

Training is school. Inference is the exam in the field. A farmer photo goes through the three levels. Then AgroScan looks up a human-written book — the Bangladesh pack — instead of letting the picture net invent a spray.

How the three disease brains vote

1. Each model outputs softmax scores (numbers that add to 1, like “percent sure”).
2. If an older checkpoint knows only 38 classes, its zeros are not allowed to water down Bangladesh-only classes.
3. Mask to the chosen crop.
4. Take the **highest-confidence single model** as `best_answer`.

The book: `data/agroscan/disease_treatments.json`

Simple: The picture net is good at “what do I see?” It is a terrible lawyer and a terrible chemist. So doses, brands, and “call 16123” come from a pack written from DAE / PPW / BRRI / BARC-style sources. 44 diseases, each in English and Bangla.

Each disease card can hold: pathogen, season, symptoms, cultural control, fertilizer (BARC FRG-style), chemical options (product, active ingredient, grams per litre, timing, max sprays, PHI — waiting days before harvest), Bangladesh brand names, organic options, safety, legal notes, and when to call the Krishi Call Center **16123**.

Many registration (AP) numbers are marked “confirm with the dealer / PPW list.” Blast and brown spot have **opposite nitrogen advice** — the app must never mix those two. That is why later retrieval is pinned to one disease when we know it.

Vision chat wrap

`POST /api/chat/vision` runs the picture pipeline, then may ask Qwen for 3–6 short farmer sentences. Qwen does *not* search the library here. It only sees the scan header plus pack steps. If Qwen repeats the same word over and over (a known 3B failure), that text is thrown away and the pack is shown instead. Sources you may see: `vision+llm`, `vision+pack`, `vision`.

Shop recommendations can match the disease to products in `data/agroscan_shop.db` (seeded from the same pack).

Talking: pack first, robot last

Simple: If you say “rice blast,” we hand you the printed page. We do not ask a chatbot to remember the dose. Chatbots invent. Printed pages do not. The talking robot is only for leftover questions like “why are my leaves sad?” when we do not have a named disease yet.

The chat decision ladder

1. **Named match** → format the pack. Source: `pack`. Qwen is not called.
2. **Same name, several crops** (e.g. “blight”) → ask which crop. Source: `clarify`.
3. **Hello / 16123 / government links / how do I use the app** → small knowledge base. Source: `kb`.
4. **Only if the KB would say something generic** → RAG + Qwen. Source: `llm`.

RAG, in kitchen language

RAG means Retrieval-Augmented Generation: first fetch pages, then let the robot talk using those pages.

AgroScan’s retrieval is **not** a magic “understands meaning” search. It is **TF-IDF** (a word-counting score: rare words that appear in both the question and a page count more) plus cosine similarity (how aligned two word-count lists are). Kid version: it looks for shared words and word pairs, not vibes.

- Build: `python -m agroscan.rag_build` writes 88 documents (44 diseases × EN+BN), each split into overview / symptoms / prevention / treatment / process.
- Index: sklearn, 20,000 word features, 1–2 word phrases, kept in RAM.
- Fetch top **6** chunks. Drop scores under 0.015. Prefer Bangla chunks if the farmer wrote Bangla. Boost treatment pages if the question says “spray” / “চিকিৎসা”.
- If we already know the disease, `allow_keys` pins retrieval so maize blight cannot leak into potato late blight.

There is no Chroma, FAISS, or embedding model. Qwen does not search. Qwen only writes.

Qwen: the talking robot (and the school it actually went to)

Simple: Qwen is a small language model that lives on this PC. It reads a recipe card we built (instructions + retrieved pages + your question) and types a reply. It is not ChatGPT in the cloud. It is not allowed to invent product names. If the retrieved pages are empty, we tell it to say “I am not sure — call 16123.”

What runs when you chat

Knob	Reality in the code
Base model	Qwen2.5- 3B -Instruct (Hugging Face, in-process Transformers)
Adapter expected	<code>llm_data/adapters/agroscan-3b</code> — not on disk, so the stock 3B Instruct talks
Adapter that exists	<code>agroscan-1.5b</code> — refused on the 3B base (wrong size)
Quantization	4-bit NF4 on CUDA, else fp16, else CPU fp32
How it writes	Greedy (always the most likely next word), max 220 tokens chat / 180 vision
When it turns on	Auto = only if a GPU is visible. Docker image is CPU PyTorch, so Qwen likely stays off there.

The extra school (LoRA) that does not yet sit in the classroom

We also built a homework pile of **69,238** Q&A lines (+ 2,000 for a quiz):

- KrishokChat ~58,300 (Bengali agri chat)
- Crop CSVs ~10,000
- AgroScan farm/disease lines a few hundred

LoRA (Low-Rank Adaptation) is a small sticker of extra weights you stick on a frozen big model so you do not retrain all billions of numbers. That sticker was trained on **Qwen2.5-1.5B** (2 epochs, batch 1, grad accum 16, lr 1e-4, max 512 tokens). Serve time uses **3B**. Different sizes — the sticker does not fit. So live RAG answers are stock 3B + retrieved pack text.

Train again with `python -m llm.build_qa_dataset` then `python -m llm.train_lora` (VS Code tasks exist). To make chat smarter, either train a 3B sticker or serve the 1.5B model with the 1.5B sticker.

Where to look, how to run, what to trust

Folder map (open I:\Agroscan\Agroscan)

If you want to...	Open
Change vision / RAG / shop / chat logic	agroscan/ + backend/main.py
Change buttons and Bangla UI	web/
Change the Android wrapper	mobile/
Edit doses and 16123 text	data/agroscan/
Retrain picture nets	python -m agroscan.train_all
Rebuild RAG pages	python -m agroscan.rag_build
Train Qwen LoRA	llm/train_lora.py
See a technical diagram	docs/AgroScan_Architecture.pdf

What a farmer actually gets today

They do this	Engine	Trust it?
Name a disease, or scan a known class	Pack (+ vision models)	This is the real product. Confirm AP numbers with a dealer when the pack says so.
Say only “blight”	Clarify UI	Yes — it asks the crop.
Open-ended chat with no disease name	TF-IDF + base Qwen 3B	Unguaranteed. Keyword search, small greedy model, no 3B LoRA attached.
Hello / helpline	Knowledge base	Yes — no robot.

Remember: AgroScan is a helper, not a replacement for an agriculture officer or veterinarian. For serious outbreaks, call **16123** (crops) or **16358** (livestock) or **999**.

Run it

1. Open the folder I:\Agroscan\Agroscan (the parent folder is only Syncthing).
2. Install PyTorch with CUDA, then pip install -r requirements.txt .
3. start_server.bat or uvicorn on port 8000.
4. For Qwen on this GPU PC: .\scripts\run_local_with_llm.ps1 (close extra browsers if Windows complains about the paging file).

Production site: <https://agroscan.mujahidulhaquejihad.com> · Docker serves CPU PyTorch (vision yes, Qwen probably off) · This booklet matches the code after the package rename to agroscan .